

One Leaky-Accumulator Cell: A Generalist Forwards-Only Rule for Temporal Credit Assignment Across Spiking Speech and EEG

Saul Dobney*

2026

Abstract

We have shown a design of analog PCN chip that learns supervised tasks *forwards-only* — no backward pass and no host in the training loop — on static images. Here we extend the same forwards-only, hardware-constrained approach to handle spiking temporal data and show credit assignment across time and demonstrate it on *two* very different spiking benchmarks — keyword spotting from audio (Google Speech Commands) and motor-imagery decoding from EEG (the THOR challenge) — with a single architecture and no per-task finessing.

Once again we find a role for a *leaky accumulator*, $x_t = \lambda x_{t-1} + u_t$, as a primary hardware element, one that can assign temporal credit through a short *local* eligibility window (a forward-mode approximation to BPTT, not a backward pass), to consolidate weights and learning. Across both tasks the design reaches about **94% of the full-BPTT (Adam) upper bound** on an identical network: 0.832 vs 0.881 on speech (0.866 once the buffer’s delay is made learnable, $\sim 98\%$), and 0.668 vs 0.709 on EEG — each with a shallow, few-step local buffer standing in for a 200–250-step backward unroll. This shows the same forwards-only, asynchronous, robust design can serve for temporal and static data based learning.

In particular, the same primitive generalises across two modalities without task-specific design, and the single setting that differs between them — a reliability-graded versus a 1-bit-sign weight write — depends only on whether temporal credit is a noisy per-timestep sum, which the EEG task independently confirms. Along the way we see the use of multiple leaky accumulators in the design, pointing to a potential central and repeatable role in analog-based learning.

The temporal learning mechanism itself is a set of bolt-on modules to the original static cell design, realised in synthesisable RTL verified *bit-exact* against a fixed-point model, including a two-layer datapath that trains both layers forwards-only; all results are pre-silicon.

1 Introduction

Parts I and II of this project built an analog predictive-coding chip that learns *in place*: a compact cell stores a weight as charge, computes in the analog domain, and is trained by a forwards-only rule that never runs a backward pass, never calibrates per device, and needs no global clock [1, 2]. We deliberately fix three design constraints — signal paths one-way (*forwards-only*), coarse and mismatched devices tolerated without calibration (*robust*), and no global clock (*asynchronous*).

*Correspondence: saul.dobney@dobney.com. This is Part III of the study; Part I [1] presents the analog cell and its unsupervised learning, and Part II [2] the forwards-only supervised architecture. Code and reproduction scripts are available at https://github.com/dobneyresearch/PCNchip_with_leakyjug_learning.

On static image classification (Part II) the rule reached backpropagation level accuracy given an identical network topology.

We now look at whether the design can assign credit *across time*, capturing and learning from spiking signals that are temporal rather than spatial. The standard approach to spiking data is backpropagation-through-time (BPTT), which uses the whole time sequence and runs the backward pass through every step to calculate the weights. However, the global, backward operations are not possible in a forwards-only hardware design. In particular, it is also physically acausal — it computes a correction at time t from information at times $t' > t$, which no forward-running substrate can supply.

Here we extend the forwards-only, hardware-constrained approach to temporal credit assignment, show that it learns, and measure it against BPTT retaining our key design constraints from Part II — one-way signal paths, tolerance of coarse and mismatched devices, no global clock, a small *local* memory per cell — and search for hardware-feasible mechanisms that fit them, judged against backpropagation aiming to see how close a realisable, forwards-only system can get to BPTT.

In solving each of the key sub-problems for this design — how to *code* time, how to *assign* temporal credit, how to *consolidate* weights, how to *grade* uncertain updates, how to *read out* a decision — we kept arriving at the same object: a **leaky accumulator**, described as a sigma-delta design in Part II,

$$x_t = \lambda x_{t-1} + u_t,$$

feeding a threshold or a bounded graded write. *One cell, several leaky accumulators reused at different leak time-constants, does all of it.* A network of these simple blocks assigns temporal credit forwards-only, reaching about 94% of full-BPTT accuracy with a shallow local buffer in place of a full backward unroll. The unification points to a common component that makes simple, naturalistic neural networks learn — reached by discovery under hardware pressure, not by aesthetic imposition.

The intuition of the leaky accumulator is that it gathers, smooths and filters signals and, in a temporal setting, retains an ‘echo’ of previous signals. Small data updates are added to the accumulator so incoming signals that are noisy and random cancel each other out; the leak acts as a decay; and threshold trigger means weights are only updated when the signal grows to become meaningful. In hardware terms this is easily constructed as a capacitor design, and is what we refer to as a ‘leaky jug’.

The strongest evidence that leaky accumulators are a core property of the mechanism, and not of one dataset, is that it holds across two temporal signals that could hardly be more different. **Google Speech Commands** (GSC) is delta-modulated audio, sparse and event-like, decoded by a deep spiking stack. **THOR EEG motor imagery** is continuous band-limited brain activity whose class information lives in signal *power*, decoded by a shallow one. The same cell, the same credit mechanism, and the same forwards-only discipline handle both. Crucially, we did not re-engineer the architecture per task: the one setting that differs between the two is not a tuned hyperparameter but a *consequence of a tested law* (Sec. 6). This is the **robustness** invariant — a generalist architecture rather than a task-specific one — made concrete.

Contributions.

- A forwards-only temporal credit-assignment rule for spiking predictive-coding networks in which each layer keeps a **short local buffer** that forms its own temporal adjoint — a *forward-mode* eligibility trace, not a backward pass — composed down the stack so a shallow buffer suffices, with a **learnable delay** (“tap”) that narrows the gap to BPTT (Sec. 4).

- A demonstration that the classifier read-out is trained by its **own on-chip gradient**, so *both* halves of learning — deep credit and read-out — are forwards-only, with no host in the loop.
- The observation that coding, credit, consolidation, reliability-grading, and read-out are **the same leaky-accumulator primitive at different timescales**, and that it is a **modular bolt-on** to the static chip of Part II, verified **bit-exact** in RTL up to a two-layer forwards-only datapath (Sec. 5).
- Evidence for a **generalist** architecture: the same rule reaches $\sim 94\%$ of full-BPTT on *two* temporal benchmarks — GSC (speech) and THOR EEG (motor imagery) — with no task-specific redesign, and the single per-task difference (graded vs 1-bit-sign write) is *predicted* by a falsifiable law that EEG independently confirms (Secs. 6–7).

1.1 Terms and notation

- **SNN** — spiking neural network. **LIF** — leaky integrate-and-fire, the standard spiking neuron: a membrane that leaks, sums input, and emits a spike when it crosses a threshold. **GSC** — Google Speech Commands, a keyword-spotting benchmark [3], here in its temporal (spiking) form under the NeuroBench protocol [4]. **EEG-MI** — electroencephalography motor imagery; the THOR challenge is a binary left/right-hand imagined-movement decode.
- **BPTT** — backpropagation-through-time: the exact gradient of a temporal network, obtained by unrolling every timestep and running the backward pass. Our upper-bound comparator, trained with the Adam optimiser.
- **Forwards-only** — credit is assigned without a global backward pass; each cell uses only quantities local to it in time and space.
- **Jug / fold** — the *leaky jug* is our shorthand for the per-synapse **leaky error accumulator** of Part II; *folding* is the periodic threshold write from the jug into the weight. We use “jug” throughout for that error accumulator specifically.
- **Eligibility trace** — a fading memory, at a synapse or neuron, of its own recent activity; the temporal analogue of the jug.
- **Rate vs leaky read-out** — two ways to turn a spike train into a decision: *rate* pools spike counts over the whole window (an aggregate); *leaky* integrates them through a leaky membrane (a genuinely temporal read-out). GSC uses the leaky read-out; the aggregate-natured EEG task uses the rate read-out; the two together make the write-rule law of Sec. 6 a testable prediction.
- **var norm (graded write)** — a reliability-graded weight update $m/\sqrt{v}$ that scales each step by how consistent the accumulated evidence has been, instead of writing a fixed ± 1 step.

2 Background and related work

We place the rule against its lineage first, because the algorithmic idea is best understood as a hardware-shaped choice among established options rather than a new invention.

2.1 The static rule, and the temporal gap

Part II’s rule is a predictive-coding *level* with separated weight and error stores (***W*** & ***E***): activations propagate forward through ***W***; a transported error δ is accumulated in a leaky error store ***E*** (the jug); and when the jug crosses a threshold, the weight moves by one code step. The error store is a first-order sigma-delta modulator [5], which is why a coarse analog cell can learn from updates

far below its resolution: it buys precision with *time* rather than with per-synapse digital memory. All of this is *static* — one forward evaluation per sample, no notion of a sequence.

A temporal task breaks that assumption such that the forward pass becomes a spiking recurrence over T timesteps, and the error observed at the read-out must be attributed to weights according to *when* their activity mattered. BPTT does this by holding the entire unrolled history and propagating the error backward through it. Three properties make that unacceptable here: it is a backward pass (forbidden on our analog path by construction); it buffers the whole sequence (a 200–250-deep store per unit); and it is acausal in time. The question of this paper is whether a *forward, local, shallow* mechanism can recover the credit that BPTT computes. Everything else in Part II — the jug, the fold, the transpose-at-source, the forwards-only discipline — is kept unchanged; only the credit path is new.

2.2 Spiking networks and how they are trained

Spiking neural networks are the natural target for analog neuromorphic hardware, a line running from Mead’s analog VLSI [6] through modern mixed-signal platforms — Loihi [7], DYNAP [8], Neurogrid [9], and the adaptive-analog tradition surveyed by Chicca et al. [10, 11]. The accuracy standard for training them is **surrogate-gradient BPTT** [12, 13]: the non-differentiable spike is given a smooth pseudo-derivative and the network is unrolled and backpropagated as an ordinary recurrent net. It is global and backward by construction, and on mismatched analog substrates is typically run with a host in the loop — exactly the arrangement a forwards-only rule removes. Our aim is to approach BPTT’s accuracy without its machinery.

2.3 Assigning temporal credit *forward* in time

Computing temporal credit without a backward pass is the province of **real-time recurrent learning** (RTRL) [14], which propagates a forward sensitivity alongside the activity but at prohibitive per-step cost. Its practical descendants trade that cost for sparsity or randomisation: UORO [15], KF-RTRL [16], and — most directly relevant — **SnAp- n** [17], the *sparse n -step* forward-influence approximation; Marschall et al. [18] give a unifying account of the family. In spiking networks the same idea appears as **e-prop** [19], which trains recurrent SNNs from local **eligibility traces** — a fading synaptic memory — and is itself an RTRL approximation of the SnAp-1 kind. Our credit path is a hardware-shaped member of this forward-mode family: a shallow, per-layer buffer that forms the eligibility trace, *composed* down the stack, and feeding a threshold or graded *hardware* write (the jug and fold) rather than a floating-point optimiser. This placement is what makes the forwards-only claim precise — the credit path is forward-mode by construction, not a truncated backward pass.

2.4 Local approximations to backpropagation

A broader literature seeks to avoid the exact backward pass or weight transport: predictive coding and its backprop-approximating variants [20, 21, 22, 23, 24], equilibrium propagation [25], and feedback alignment [26, 27]. These share our aim but are largely developed on static tasks, or — in the case of equilibrium propagation — reintroduce an iterative relaxation this design avoids. We retain the *true* transpose (computed at the chip that owns the weight, so it costs no weight transport) and add to that lineage a *temporal* credit path that stays forward-mode and local. Finally, the leaky-accumulator-of-outer-products at the heart of that path is structurally a fast-weight / linear-attention memory; that connection points to attention as a **parallel track** in the same cell family, pursued separately.

3 The learning rule: one leaky accumulator, reused

This section states the rule as an algorithm, independent of how it is built; the hardware follows in Sec. 5. It is organised around a single primitive and the several roles it plays.

3.1 The primitive

The load-bearing component, used in several places, is in every case a leaky accumulator: a state that decays by a factor λ each step and is topped up by an input,

$$x_t = \lambda x_{t-1} + u_t. \quad (1)$$

For $\lambda \rightarrow 1$ it is a pure integrator (a running sum); for $\lambda < 1$ it is an exponentially-weighted memory with time constant $1/(1 - \lambda)$. The whole design uses multiple variations of this cell with different leaks and different inputs, sometimes read through a threshold, sometimes through a bounded graded write. Read one way it is a rolling sum that smooths a noisy stream; read another it triggers a discrete event when its total crosses a level. We take the roles in turn.

3.2 Coding: the LIF membrane

The input is delta-modulated into spikes, and each hidden layer is a layer of leaky integrate-and-fire (LIF) neurons in the standard SNN convention. A neuron’s membrane is (1) with leak α and the layer’s weighted spike input; it fires when the membrane crosses a threshold ϑ and subtracts it (a soft reset):

$$\mathbf{a}_{\ell,t} = \alpha \mathbf{a}_{\ell,t-1} + \mathbf{W}_\ell \mathbf{s}_{\ell-1,t}, \quad \mathbf{s}_{\ell,t} = \mathbf{1}[\mathbf{a}_{\ell,t} \geq \vartheta], \quad \mathbf{a}_{\ell,t} \leftarrow \mathbf{a}_{\ell,t} - \vartheta \mathbf{s}_{\ell,t}. \quad (2)$$

The membrane leak $\alpha \approx 0.9$ gives a fast (~ 10 -step) memory: it is what lets a spike influence the layer’s output for a short window after it arrives. This is the leaky-accumulator primitive in its fastest role, coding the passage of time into a decaying trace.

3.3 Temporal credit: a per-layer eligibility window

The credit problem is to know, at the moment an error is observed, which past activity to reward. BPTT does this by propagating the error backward through the whole unrolled sequence. We answer it *forward and locally*, drawing on e-prop [19] and the RTRL/SnAp family [14, 17, 18].

Each neuron carries a **readiness** $\psi_{\ell,t}$ — the surrogate derivative of its spike, large only when its membrane sits near the threshold of (2) — and receives the error $\mathbf{e}_{\ell,t}$ arriving at its layer (from the read-out at the top, or routed down from above). The layer’s local **adjoint** is the readiness-gated error summed over a short window,

$$\delta_{\ell,t} = \sum_{k=0}^n w_k \psi_{\ell,t+k} \odot \mathbf{e}_{\ell,t+k}, \quad n \approx 8, \quad (3)$$

realised by a hardware-credible **n -deep buffer of timesteps, held at the cell/layer level**. The buffer allows us to mimic the forwards-in-time calculations of BPTT, acting as a time-delay window which makes $\delta_{\ell,t}$ available n steps late, a fixed learning delay rather than a look-ahead. The weights w_k over the window are the **tap weights** (Sec. 4.3): setting them all to 1 recovers a uniform box; making them learnable lets the buffer discover which delays carry credit. This adjoint does two jobs. It is **routed to the layer below** by the transpose of the layer’s *own* weights,

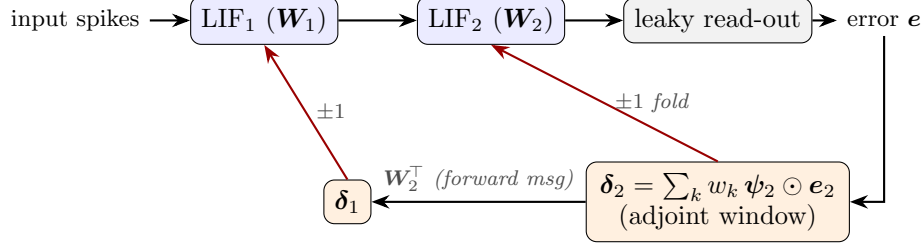


Figure 1: The temporal credit loop, forwards-only. Forward: input spikes $\rightarrow$ LIF stack $\rightarrow$ leaky read-out $\rightarrow$ error e . Each layer forms its adjoint $\delta_\ell = \sum_k w_k \psi_\ell \odot e_\ell$ (a windowed, tap-weighted sum of readiness-gated error, (3)); the adjoint both writes its own weights through the leaky error accumulator (the ± 1 fold) and is relayed to the layer below by the transpose $\mathbf{W}_\ell^\top$ — a forward-directed message about the past, not a reverse channel. No analog signal flows backward.

$e_{\ell-1,t} = \mathbf{W}_\ell^\top \delta_{\ell,t}$ — a forward-directed message computed on the chip that owns $\mathbf{W}_\ell$, not a reverse channel — carrying credit down the stack; and it drives that layer’s weight update by pairing with the **current** presynaptic spike (Sec. 4.4).

Equation (3) is a *local eligibility trace* in the sense of e-prop [19], and formally a **sparse n -step forward-sensitivity approximation** in the RTRL family — essentially SnAp- n [17] — formed *forward* in time by the buffer, and never requiring a signal to travel backward across the network. The exact truncated-BPTT adjoint would additionally weight the window terms by a decay $\prod_j \alpha(1 - \partial \psi_{\ell,t+j})$; we find the *uniform* window ($w_k \equiv 1$) performs identically at $n \approx 8$ (Sec. 6), so the simpler box is the hardware default. Two facts make a *shallow* buffer enough. First, the membrane leak α has already discarded activity older than ~ 10 steps, so credit beyond the window has decayed anyway. Second, the per-layer adjoints **compose down the stack**: each layer’s routed error already carries the buffered contribution of the layers above, so the *effective* temporal reach is deeper than any single buffer while each stays at $n \approx 8$. Sec. 6 shows n between 6 and 12 is indistinguishable — a per-layer, forward, ~ 8 -deep buffer standing in for a 200–250-step backward unroll.

Making the tap weights w_k *learnable* is a small addition with a real payoff. A learnable per-delay weighting (a short causal FIR over the buffer, initialised to the identity so training begins exactly at the uniform box) lets the network place credit at the delays that matter. On speech this recovers the last few points to BPTT (Sec. 6); on EEG, whose informative timescale is longer, the *reach* of the window is itself load-bearing — extending it from 8 to 16 taps is what lifts the forwards-only rule to its $\sim 94\%$ mark while the BPTT comparator is indifferent to it. The tap is best read as a **controller for the time buffer**: the buffer supplies the reach, the taps decide how to spend it.

3.4 Consolidation, and reliability-graded writing

Each layer’s adjoint pairs with the *current* presynaptic spike $z_{\ell-1,t} = s_{\ell-1,t}$ to form the per-sample weight gradient $\mathbf{g}_\ell = \sum_t \delta_{\ell,t} z_{\ell-1,t}^\top$ (the windowing lives in the adjoint; the presynaptic term is the spike at time t). This is written to the weights through the same jug and fold as Part II — another leaky accumulator, now with a threshold. The jug integrates credit; every F samples the fold moves the weight and resets the jug:

$$\mathbf{E}_\ell += \mathbf{g}_\ell, \quad \text{every } F \text{ samples: } \mathbf{W}_\ell += \eta \text{sign}(\mathbf{E}_\ell), \quad \mathbf{E}_\ell \leftarrow 0. \quad (4)$$

The jug is again a leaky accumulator, now at the *slowest* timescale (many samples per fold) — the consolidation of transient credit into a durable weight. The ± 1 **sign write** of (4) (i.e. $+1$

or -1) is the base rule, and on tasks whose credit is an *aggregate* it is all that is needed — as we will see, it is what the EEG task uses. On the GSC temporal task it is not enough, because the per-timestep credit in (3) is a *noisy sum*: individual timesteps disagree, and a bare sign discards how *consistent* the evidence was. There we replace the sign with a reliability-graded step — itself two more leaky accumulators, of the credit’s mean and of its energy:

$$\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1) \mathbf{E}, \quad \mathbf{v} \leftarrow \beta_2 \mathbf{v} + (1 - \beta_2) \mathbf{E}^2, \quad \mathbf{W} += \eta \frac{\mathbf{m}}{\sqrt{\mathbf{v}} + \epsilon}, \quad (5)$$

which scales each synapse’s step by how reliably its evidence has pointed the same way. It is self-scaling and bounded (unlike a raw-magnitude write, which overfits), and — crucially — it earns its keep *only* when credit is a noisy per-timestep sum.

3.5 On-chip read-out: the second half, also forwards-only

Credit assignment in the hidden layers is the first half of the learning. In addition, the classifier read-out is the second half. On the static chip the classifier was fit off-line by a host least-squares step. On these temporal tasks we also train the classifier on-chip by its own gradient. Consequently, the read-out classifier sees a feature vector $\mathbf{f}$ — for GSC a *leaky-integrated* one, (1) once more; for EEG an aggregate spike *rate* — and updates by the output error:

$$\mathbf{e} = \text{onehot}(y) - \hat{\mathbf{p}}, \quad \mathbf{W}_f += \eta_f \mathbf{e} \mathbf{f}^\top, \quad \mathbf{b}_f += \eta_f \mathbf{e}. \quad (6)$$

This *cold* read-out (trained from scratch alongside the hidden layers, rather than refit each epoch) both closes the gap a periodically-refit read-out leaves and removes the host from the read-out. It needs one accommodation — a learning-rate bootstrap, since a cold head must move faster than the slowly-consolidating hidden weights to keep up — after which both halves of learning are the same forward, local machinery. The single fold pulse-rate that governs consolidation (the parameter we call η / boss-lr) is, notably, the *same value* across both benchmarks and the earlier static task: a transferable constant rather than a per-task setting (Sec. 7).

3.6 One cell, many timescales

Collecting the roles, the design is several single leaky accumulators specialised only by their leak and input. This points to the leaky accumulator being a fundamental primitive for hardware-constrained neuromorphic networks of this kind:

Table 1: The same leaky-accumulator primitive (1) at five timescales. Coding, credit, consolidation, grading and read-out are not five mechanisms but one cell reused — mirroring the way biology reuses leaky integration from membrane to eligibility to homeostasis.

role	equation	leak / timescale	in the network
temporal coding	(2)	$\alpha \approx 0.9$, ~ 10 steps	LIF membrane
leaky read-out	(6)	$\beta \approx 0.9$, ~ 10 steps	dendritic integration
feature temporal credit	(3)	window $n \approx 8\text{--}16$	eligibility trace

role	equation	leak / timescale	in the network
weight consolidation	(4)	many samples per fold	synaptic tag $\rightarrow$ consolidation
reliability grading	(5)	$\beta_2 \approx 0.99$, slowest	homeostatic scaling

4 Hardware realisation as a modular upgrade

The base of the design is unchanged from the static cell of Part II — the leaky jug. Because none of the temporal elements touch that base, and because each is a version of the same primitive the chip already implements, the temporal capability can be considered as a set of *bolt-on modules*, not a redesign. Each can be included or omitted at the cell/chip/layer level per task, and that separability is what keeps the design modular and the key arithmetic at the neuron level, where it scales.

- **The LIF membrane** (2) is a leaky accumulator with a threshold — the same structure as the jug, applied to activations rather than errors. Bypassed, the cell is the static analog MAC of Part II.
- **The per-layer credit buffer** (3) is a small local first-in-first-out store (depth $n \approx 8\text{--}16$) holding the recent readiness-gated error that forms the adjoint. It is **per-neuron, not per-synapse** — a few hundred bits per cell — and, with a 1-bit sign fold, the temporal upgrade adds *no* per-synapse digital state, preserving the base jug’s defining property. The presynaptic spike is paired at the write, and the transpose that routes the adjoint down is the existing Part-II engine. At depth 0 the cell reverts to the static update.
- **The tap weights** are a short causal FIR (w_k of (3)) over that same buffer — a handful of shared coefficients per cell, not per synapse. Delta-initialised to the identity, the module starts as the uniform box and is trained by the same fold rule; it is the cheapest of the temporal additions and, on both tasks, the one that buys reach toward BPTT.
- **The graded write** (5) is the one module that *does* add per-synapse state: two slow accumulators (m, v), several times the weight code. It is therefore **optional** — it buys the last few points on the tasks that need it and nothing on the tasks that do not — and its $\sqrt{\cdot}$ and divide are one *shared, swept* unit, as the jug shares its comparator, not per-synapse logic. Whether a task needs it at all is *predicted* by Sec. 6, so it is enabled by a rule, not a sweep.
- **The read-out runs per sample, not per timestep**: it acts on the pooled feature, so its whole cycle (score, error, gradient (6), and the top- δ transpose) sits *outside* the per-timestep loop at $1/T$ its rate, and can be a plain digital MAC; the cold gradient moves it on-chip from the host.

What stays in firmware is only what stayed in Part II: the soft-max/error scalar at the read-out. Everything else — coding, credit, consolidation, grading, read-out — is the on-chip primitive. A *tiered* hardware story follows directly from the testing in Sec. 6: the cheapest configuration (a spike-counter read-out with a 1-bit sign write) already reaches 0.78 on GSC; adding the leaky read-out, the graded write, and the learnable taps buys the rest. The design lets the integrator choose where to sit on that curve, per task.

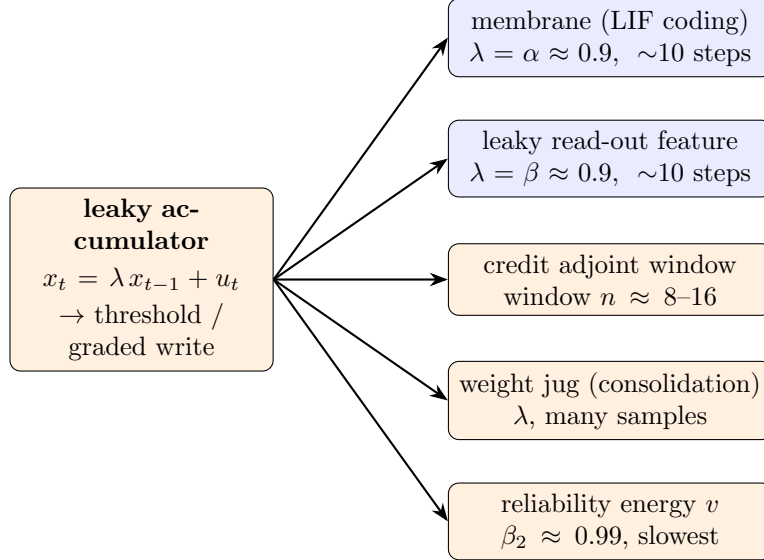


Figure 2: One cell, five timescales. The same leaky accumulator $x_t = \lambda x_{t-1} + u_t$ followed by a threshold or graded write, specialised only by its leak λ and its input, performs temporal coding, leaky read-out, temporal credit, weight consolidation, and reliability grading. The unification is a finding of the search, not an imposition.

4.1 A bit-faithful RTL implementation

The modular hardware argument has also been tested by implementing the temporal datapath in synthesisable register-transfer-level (RTL) Verilog, with each block verified **bit-exact** against a fixed-point reference model — the same double-build discipline Part II used for the static jug, on the same open 130 nm base and simulation harness.

Every element has a verified module: the **LIF membrane** with its readiness output (2); the **adjoint window** that computes the credit of (3); the **graded write** (5), whose $m/\sqrt{v}$ is a floor integer square-root and divide (a look-up reciprocal-root is the area-optimal alternative); the **cold read-out** accumulate-and-write (6); and the **transpose hop** $W_\ell^\top \delta_\ell$ that routes credit down a layer. For each, a fixed-point Python model is the golden reference and a testbench checks every output bit-for-bit; the whole set runs as a pass/fail regression in which silence counts as failure.

Three integration levels close the loop from block to chip, each verified **cycle-accurate**. A **single-synapse lane** wires the forward LIF, the adjoint window, the presynaptic-spike delay, the credit accumulator, and the graded-write fold into one streaming datapath over a full input sequence. An **array tile** replicates the lane across several post- and pre-synaptic units, sharing the presynaptic lines — the width dimension. And a **two-layer datapath** demonstrates the paper’s central claim in silicon-ready form: the top error trains *both* layers with no backward pass, because each layer’s adjoint both writes its own weights (through the jug fold) and is routed to the layer below through the transpose. Blocks and integration alike match their references bit-exact.

This turns “hardware-feasible” from an argument into a measured result with datapaths verified at the block, tile, and two-layer level, though not yet as a full swept array; resource *sharing* — one comparator and one square-root/divide time-multiplexed across the array — is an area optimisation on top of the proven datapath. Transistor-level SPICE of the new cells and place-and-route remain, as in Part II, the pre-silicon steps beyond this study.

5 Evaluation on two temporal benchmarks

5.1 Setup

The design was tested using the *same* architecture on two temporal signals of opposite character, without per-task finessing of the rule parameters.

Google Speech Commands (GSC), under the NeuroBench temporal protocol [3, 4]: 35 keyword classes, audio delta-modulated into spike trains of $T = 200$ timesteps, a [20, 256, 256, 256] LIF network, and a **leaky-integrator read-out**. The signal is sparse and event-like, and its credit is a genuinely temporal per-timestep sum.

THOR EEG motor imagery (EEG-MI): binary left/right imagined-hand movement, 62 EEG channels over $T = 250$ timesteps, decoded by a shallow spiking network with a **rate (spike-count) read-out**. Here the class information is band *power* — a second-order, aggregate quantity — so the credit at a synapse is a pooled count rather than a noisy per-step sum. The contrast with GSC is deliberate: it is what makes the write-rule law testable rather than asserted.

For both, the comparator is the *same forward network* trained end-to-end by full BPTT with the Adam optimiser — the unconstrained upper bound. The forwards-only rule and the BPTT comparator differ only in how credit is assigned and written; the forward pass, the data, and the read-out target are identical. We report best validation accuracy over seeds; recipes and seeds are in the repository. The framing is deliberately asymmetric: Adam-BPTT is not a rival to beat but the *best pure-maths result* against which a deliberately constrained system is measured. The claim is not equality but *how close a realisable, forwards-only system gets*.

5.2 Speech: the result across scale

Table 2: Forwards-only vs full-BPTT on temporal GSC (leaky read-out), across training-set size, and with the learnable-tap buffer at full scale.

training data	forwards-only rule	Adam (full-200-step BPTT)	gap
8k / 12 ep	0.668	0.721	0.054
30k / 20 ep	0.770	0.806	0.036
full 84k / 20 ep	0.832	0.881	0.049
full 84k, + learnable taps	0.866	0.881	0.015

The forwards-only rule reaches 0.832 on the full benchmark against a full-BPTT ceiling of 0.881 — about 94% of the backpropagation result, forwards-only, with a shallow buffer against a 200-step unroll. This headline uses the committed *uniform* window; the faithful decaying adjoint reaches the same 0.831, so the number rests on the mechanism the hardware actually implements. Making the buffer’s delay **learnable** (the taps of Sec. 4.3) then adds +0.032 at full scale and *stacks on* the uniform cell, reaching 0.866 and narrowing the gap to Adam from 0.049 to 0.015 — about 98% of BPTT. The tap replaces nothing; it lets the same buffer spend its reach where credit lies.

5.3 EEG: the same rule, a second modality

Table 3: Forwards-only vs full-BPTT on THOR EEG-MI (rate read-out, 4 seeds). The forwards-only rule *needs* the wider window; the BPTT comparator is indifferent to it.

window depth (tap K)	forwards-only rule	Adam (full BPTT)	gap
$K = 8$	0.637	0.705	0.068
$K = 16$	0.668 ± 0.001	0.709 ± 0.005	0.042

On EEG the forwards-only rule reaches 0.668 against an Adam-BPTT ceiling of 0.709 (chance 0.50) — again about 94% of the backpropagation result (94.1%), and a third independent confirmation of the $\sim 94\%$ figure after EMNIST (static, Part II) and GSC (temporal). Two features are worth drawing out. First, the **buffer reach is load-bearing here**: extending the window from $K = 8$ to $K = 16$ lifts the rule by +0.030 while the BPTT comparator barely moves ($0.705 \rightarrow 0.709$) — the forwards-only rule is the one that must *reach* far enough, exactly as the composition argument predicts, because EEG’s informative timescale is longer than speech’s. Second, this task uses the **1-bit sign write**, not the graded one, and does so by prediction rather than by choice (next section).

5.4 What is load-bearing, and the write-rule law

Two mechanisms carry the temporal result, and both are settled by testing rather than assertion.

The learnable tap weights / buffer reach. On GSC the learnable tap weights are worth +0.032 (uniform 0.834 $\rightarrow$ tapped 0.866); on EEG the window depth is worth +0.030 ($K8 \rightarrow K16$). On both, buffer *depth* is otherwise flat between 6 and 12 uniform steps, and the uniform box matches the exact decaying adjoint over 3 seeds (+0.004, not significant) — so the hardware implements the simpler box and spends its parameters on the taps, which is where reach is actually bought.

The graded write, and its falsifiable law. On GSC’s leaky read-out, replacing the graded write (5) with a bare sign costs 12 pp ($0.649 \rightarrow 0.770$, positive in every seed, $p \approx 0.004$) — it is load-bearing. Our account is that grading helps *precisely* when credit is a noisy sum of per-timestep terms. That account predicts it should *stop* helping wherever credit is an aggregate. Two independent controls confirm it: a *rate* read-out on GSC (which pools spikes into one count per synapse) sees only +0.014 from grading, not significant; and the **EEG task**, whose rate read-out is aggregate by nature, sees *no* benefit and therefore uses the sign write. The single setting that differs between our two benchmarks is thus not a tuned knob but the output of a tested law:

Table 4: The write-rule law. Whether a task needs the graded write is predicted by how its credit is formed, and the two benchmarks fall on opposite sides of the prediction.

credit signal	example	graded write helps?
noisy per-timestep sum	GSC, leaky read-out	yes (+0.121, $p \approx 0.004$)
aggregate (pooled)	GSC, rate read-out	no (+0.014, n.s.)
aggregate (pooled)	EEG-MI, rate read-out	no (sign write used)

6 Robustness: one generalist cell, not a task-specific design

Of the three invariants inherited from Parts I and II — forwards-only, robust, asynchronous — it is *robustness* that the temporal work most sharpens, and it is the result we most want to carry. Robustness here means more than tolerance of device mismatch (which Part II established, and which the coarse 1-bit write and per-synapse learning-rate spread still provide). It means a **generalist** architecture: one cell, one credit mechanism, one forwards-only discipline, applied to two temporal signals of opposite character without redesign.

The evidence is that the design does *not* get re-tuned per task. GSC is deep, sparse and event-like with a leaky read-out; EEG is shallow, continuous and power-based with a rate read-out. The forward model, the eligibility-window credit path, the transpose routing, the jug-and-fold consolidation, the cold on-chip read-out, and the fold pulse-rate constant are common to both. The learnable-tap buffer is the shared mechanism that supplies temporal reach on each. Where the tasks differ, they differ in ways the architecture *anticipates* rather than requires hand-setting:

- The **write rule** (graded vs 1-bit sign) is the one substantive difference, and it is *predicted* by the write-rule law: GSC’s noisy per-timestep credit calls for grading, EEG’s aggregate credit does not. This is the opposite of task-specific parameter finessing — the same rule, applied, tells you which module to enable.
- The **fold pulse-rate** is a single value transferred unchanged across GSC (temporal), EMNIST (static, Part II), and EEG (band-power). For a substrate that cannot trim per device, a transferable global constant with a broad tolerance plateau is exactly the property a generalist design needs; a knob that had to be re-found per task would undermine the whole approach.

We read this as the practical meaning of a hardware-first design: because the cell is chosen to satisfy fixed physical constraints rather than to fit one dataset, it inherits a generality that a task-optimised network does not. The leaky accumulator is the prime component in this because it is the one part reused everywhere — and reuse across roles *within* a task is what makes reuse across tasks unsurprising. A generalist edge learner does not need a new architecture for every signal; it needs one robust primitive and a rule for where to apply it.

7 Approaching BPTT: what the gap is, and why it is acceptable

The buffer (3) recovers the BPTT gradient *direction* closely — an independent probe finds cosine similarity near unity — so it is fair to ask whether it is simply truncated BPTT wearing a forward mask. It is not, and the distinction is physical. In the taxonomy of online learning [18] the buffer is a *forward-mode* (RTRL-family) approximation — SnAp- n [17] — not truncated BPTT: the two converge to the same gradient but compute it in opposite directions, and only the forward one is realisable on this substrate. The buffer is a *per-layer, local* accumulation computed *forward in time*: no error propagates backward across layers, and no global unrolled history is held. Its agreement with BPTT is the useful finding — that a shallow local eligibility trace, composed down the stack, is a good approximation to the global gradient — not a contradiction.

We also looked hard for the residual. Across both tasks a ~ 5 -point gap to Adam remains at the uniform-window setting (GSC 0.832/0.881; EEG 0.668/0.709), and we searched the write path for it: finer per-synapse magnitude writes, a self-annealing step-size damper, and the graded write itself all bear on *how* credit is written. None closes the gap, and the reason is instructive: the graded write is the strongest bounded write in the family, and where it helps (GSC) it still leaves the same residual, while where it does not (EEG) a finer write is not the lever either. The residual

is therefore not in the *magnitude* of the written credit but in its *direction* — the forwards-only windowed adjoint computes a gradient that approximates BPTT’s but is not identical to it, by design (a bounded forward reach in place of the full backward unroll). On GSC the learnable taps recover most of it (to 0.866, $\sim 98\%$); on EEG, whose credit direction is genuinely harder to form forward, more of it remains. This is the honest boundary: a close, cheap, forward *approximation* to BPTT, not an equivalence, with a residual that is bounded and localised rather than mysterious.

That boundary is, we think, the right one for a hardware-first design to accept. A maths-perfect BPTT result is not necessarily physically reachable on a forward, local, coarse substrate, and chasing the last points by adding a backward channel or unbounded per-synapse memory would forfeit the very properties the design exists to provide. A robust generalist that reaches $\sim 94\text{--}98\%$ of the backpropagation ceiling on two unlike temporal tasks, forwards-only and with a few hundred bits of local buffer, is the useful object. We assume Nature does not compute a perfect gradient either; it computes a good-enough one, locally and forward, and so do we.

8 Limitations and conclusion

Limitations and future work. All results are pre-silicon (behavioural and bit-faithful models, not a fabricated die). The temporal rule is now characterised on two benchmarks rather than one, which broadens the generality claim, but both are edge-scale classification; whether the primitive’s reuse transfers to larger or more structured temporal problems is open. Two trade-offs are characterised but left open. The **~ 5 -point gap to full BPTT** at the uniform setting is closed to ~ 1.5 points on GSC by the learnable taps but remains wider on EEG; we have localised it to the forward credit *direction* rather than the write magnitude, but not eliminated it. And the **graded write’s per-synapse cost**: it is the one module that adds per-synapse digital state, for the tasks the law says need it; whether a cheaper middle ground between a 1-bit sign and full per-synapse reliability recovers most of that gain is flagged, not resolved. The credit buffer is otherwise a truncated local adjoint (credit outside its window is not recovered; the composition argument that keeps it shallow is empirical, not bounded in general).

Conclusion. A forwards-only, hardware-constrained learning rule can assign temporal credit on real spiking benchmarks, reaching about 94% of full-BPTT accuracy — up to $\sim 98\%$ with a learnable-delay buffer — with a shallow local buffer and no backward pass. The economy of the result is one of its points: coding, credit, consolidation, reliability grading, and read-out are not separate mechanisms but one leaky-accumulator cell reused at several timescales, and that cell is a modular addition to the static chip rather than a new design. But the point we most want to leave is *robustness*. The same architecture handles two temporal signals of opposite character — sparse audio spikes and continuous EEG power — without task-specific redesign, and the single setting that differs between them is predicted by a falsifiable law rather than tuned. The unification of roles was not imposed but found; the generalisation across tasks was tested, not hoped for; and both, we read, are properties of building from a robust primitive under fixed hardware constraints — and of the biology that builds under the same ones.

Acknowledgements

Architectural direction and design decisions are the author’s. Simulators and experiments were implemented with Claude Code [28] (Opus and Fable models), whose contribution was the rate at which the mechanism space could be searched and characterised.

References

- [1] S. Dobney, “Scalable modular analog VLSI for predictive coding networks: Temporal multiplexing, learned routing, and zero weight bandwidth,” *arXiv preprint*, 2026, companion paper (Part I); design files at https://github.com/dobneyresearch/PredictiveCodingNetworks_AnalogVLSIdesign.
- [2] —, “A forwards-only, asynchronous analog architecture for predictive-coding networks,” 2026, part II; companion architecture paper.
- [3] P. Warden, “Speech commands: A dataset for limited-vocabulary speech recognition,” *arXiv preprint arXiv:1804.03209*, 2018.
- [4] J. Yik *et al.*, “NeuroBench: A framework for benchmarking neuromorphic computing algorithms and systems,” *Nature Communications*, vol. 16, 2025.
- [5] P. O’Connor and M. Welling, “Sigma delta quantized networks,” *arXiv preprint arXiv:1611.02024*, 2016.
- [6] C. Mead, *Analog VLSI and neural systems*. Addison-Wesley, 1989.
- [7] M. Davies, N. Srinivasa, T.-H. Lin *et al.*, “Loihi: A neuromorphic manycore processor with on-chip learning,” *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.
- [8] S. Moradi, N. Qiao, F. Stefanini, and G. Indiveri, “A scalable multicore architecture with heterogeneous memory structures for dynamic neuromorphic asynchronous processors (DYNAPs),” *IEEE Transactions on Biomedical Circuits and Systems*, vol. 12, no. 1, pp. 106–122, 2018.
- [9] B. V. Benjamin, P. Gao, E. McQuinn, S. Choudhary, A. R. Chandrasekaran, J.-M. Bussat, R. Alvarez-Icaza, J. V. Arthur, P. A. Merolla, and K. Boahen, “Neurogrid: A mixed-analog-digital multichip system for large-scale neural simulations,” *Proceedings of the IEEE*, vol. 102, no. 5, pp. 699–716, 2014.
- [10] E. Chicca, F. Stefanini, C. Bartolozzi, and G. Indiveri, “Neuromorphic electronic circuits for building autonomous cognitive systems,” *Proceedings of the IEEE*, vol. 102, no. 9, pp. 1367–1388, 2014.
- [11] A. Rubino, M. Cartiglia, M. Payvand, and G. Indiveri, “Neuromorphic analog circuits for robust on-chip always-on learning in spiking neural networks,” in *IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)*, 2023.
- [12] E. O. Neftci, H. Mostafa, and F. Zenke, “Surrogate gradient learning in spiking neural networks,” *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.
- [13] B. Cramer, S. Billaudelle, S. Kanya, A. Leibfried, A. Grübl, V. Karasenko, C. Pehle, K. Schreiber, Y. Stradmann, J. Weis, J. Schemmel, and F. Zenke, “Surrogate gradients for analog neuromorphic computing,” *Proceedings of the National Academy of Sciences*, vol. 119, no. 4, p. e2109194119, 2022.
- [14] R. J. Williams and D. Zipser, “A learning algorithm for continually running fully recurrent neural networks,” *Neural Computation*, vol. 1, no. 2, pp. 270–280, 1989.

- [15] C. Talleg and Y. Ollivier, “Unbiased online recurrent optimization,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [16] A. Mujika, F. Meier, and A. Steger, “Approximating real-time recurrent learning with random Kronecker factors,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.
- [17] J. Menick, E. Elsen, U. Evci, S. Osindero, K. Simonyan, and A. Graves, “Practical real time recurrent learning with a sparse approximation,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [18] O. Marschall, K. Cho, and C. Savin, “A unified framework of online learning algorithms for training recurrent neural networks,” *Journal of Machine Learning Research*, vol. 21, no. 135, pp. 1–34, 2020.
- [19] G. Bellec, F. Scherr, A. Subramoney, E. Hajek, D. Salaj, R. Legenstein, and W. Maass, “A solution to the learning dilemma for recurrent networks of spiking neurons,” *Nature Communications*, vol. 11, no. 1, p. 3625, 2020.
- [20] R. P. Rao and D. H. Ballard, “Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects,” *Nature neuroscience*, vol. 2, no. 1, pp. 79–87, 1999.
- [21] K. Friston, “The free-energy principle: a unified brain theory?” *Nature Reviews Neuroscience*, vol. 11, no. 2, pp. 127–138, 2010.
- [22] J. C. Whittington and R. Bogacz, “An approximation of the error backpropagation algorithm in a predictive coding network with local Hebbian synaptic plasticity,” *Neural computation*, vol. 29, no. 5, pp. 1229–1262, 2017.
- [23] B. Millidge, A. Seth, and C. L. Buckley, “Predictive coding: a theoretical and experimental review,” *arXiv preprint arXiv:2107.12979*, 2022.
- [24] R. Bogacz, “A tutorial on the free-energy framework for modelling perception and learning,” *Journal of mathematical psychology*, vol. 76, pp. 198–211, 2017.
- [25] B. Scellier and Y. Bengio, “Equilibrium propagation: Bridging the gap between energy-based models and backpropagation,” *Frontiers in Computational Neuroscience*, vol. 11, p. 24, 2017.
- [26] T. P. Lillicrap, D. Cownden, D. B. Tweed, and C. J. Akerman, “Random synaptic feedback weights support error backpropagation for deep learning,” *Nature Communications*, vol. 7, no. 1, p. 13276, 2016.
- [27] A. Nøkland, “Direct feedback alignment provides learning in deep neural networks,” in *Advances in Neural Information Processing Systems*, vol. 29, 2016.
- [28] Anthropic, “Claude Code,” <https://claude.ai/code>, 2025, aI-assisted software engineering tool.